IUT Montpellier-Sète
Département Informatique
M. Gasquet, G. Hisler, S. Robillard, P. Valicov

Diagramme de classes de conception

Table des matières

1 Introduction **2**

2 Diagramme de classes de conception **2**

3 Modélisation des classes **3**

4 Associations entre les classes **3**
4.1 Navigabilité, attributs 4
4.2 Collections 5
4.3 Classes-associations 5
4.4 Associations qualifiées 6
4.5 Composition 7

5 Énumérations **8**

6 Héritage **9**

7 Interfaces **9**

8 Généricité **10**

9 Threads **11**

10 Les dépendances **11**

11 Architecture en couches **15**

12 Qualité de la conception **18**
12.1 Les principes SOLID 18
12.2 Les designs patterns 18
12.3 Les frameworks 19

13 Ressources supplémentaires **19**

1 Introduction

Le cours de qualité de développement vise à étudier et maîtriser les différents concepts liés à la phase de **conception** et également les **bonnes pratiques** et les outils visant à améliorer le code produit et l'architecture générale du logiciel afin d'obtenir un projet extensible, modulable, adaptable qui sera facile à faire évoluer.

Lors du cours d'**analyse** vous avez appris à maîtriser les outils vous permettant de modéliser le **besoin** du client sous la forme de divers diagrammes afin de comprendre ce que devait faire le système informatique cible. Dans ce premier cours, nous allons donc étudier la seconde étape : la **conception** qui va détailler la manière dont ce système va être implémenté, au niveau logiciel. On rentre donc dans une partie technique où le client n'intervient pas, contrairement à l'analyse.

L'objectif de la conception est de modéliser à l'aide de différents diagrammes (UML) l'architecture et le comportement de notre logiciel, en terme d'objets. Pour commencer, nous allons donc simplement voir comment utiliser les diagrammes UML en conception. Cependant, si dans un premier temps l'objectif va être de maîtriser la modélisation et la sémantique de ces diagrammes, le véritable but ultime sera de produire une **bonne conception** en faisant des choix judicieux !

Retenez simplement que ce n'est pas parce qu'on sait dessiner des diagrammes que l'architecture et le fonctionnement du logiciel est de qualité ! Nous verrons d'ailleurs dans de prochains cours comment ces choix peuvent influencer positivement ou négativement l'évolutivité du projet.

2 Diagramme de classes de conception

Le diagramme de classes de conception est généralement construit à partir du diagramme de classes persistantes, produit en **analyse**. En termes d'architecture, ces classes se retrouvent dans la couche **business** (métier) de notre logiciel, voire au niveau de la sous-couche **entity**.

Durant la phase de conception, on vient ajouter toutes les autres classes qui composeront les **couches** de notre logiciel :

- Les classes relatives à l'**IHM** (couche **présentation**).
- Les classes de **contrôle** (contrôleurs) au niveau de la couche **application** qui contrôlent les données reçues depuis l'IHM.
- Les classes contenant la **logique métier** de l'application et qui manipulent directement les entités métiers (couche **service**, moteur/coeur de l'application).
- Les classes relatives à la persistance des entités (chargement / déchargement) au niveau de la couche **stockage**.
- Les classes relatives au transport des données (si besoin) au niveau de la couche **réseau**.

Il est à noter que certaines de ces couches sont optionnelles. Par exemple, certains programmes n'ont pas besoin de transporter des données en réseau. Certains programmes n'ont pas besoin d'avoir des données persistantes. Aussi, tout dépend du modèle d'architecture retenu pour développer l'application. Nous parlerons plus amplement de ces différentes couches à la fin du cours.

Contrairement à la phase d'analyse, nous allons définir le **comportement** de chaque classe en spécifiant les différentes méthodes (et constructeurs) de chaque classe de notre diagramme.

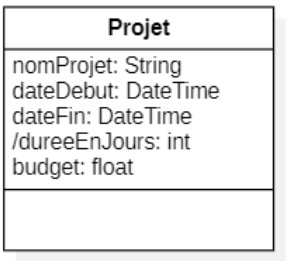
On peut produire ce diagramme en **pro-ingénierie** (diagramme puis code) afin de déterminer à l'avance l'architecture de notre logiciel ou bien en **retro-ingénierie** pour analyser l'architecture d'un logiciel existant. Bien sûr, ce diagramme peut évoluer pendant la conception, mais dans le futur, nous verrons qu'une bonne conception implique peu (voire pas) de modification de l'existant, seulement des ajouts et des extensions.

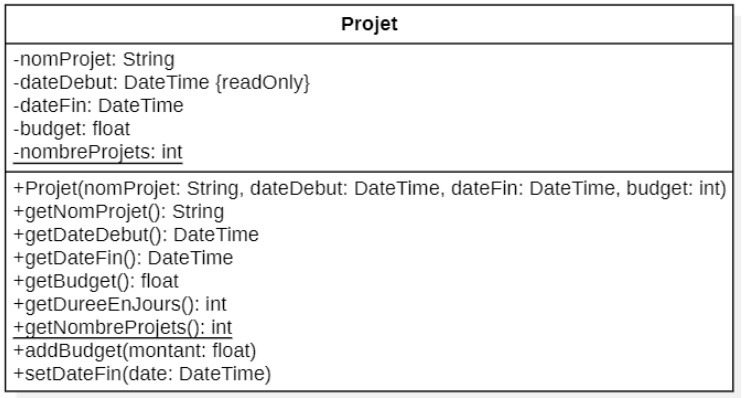
Certains outils (comme StarUML) permettent de générer le diagramme de classes d'un logiciel existant et inversement (génération du squelette du programme). Dans l'IDE **IntelliJ**, un plugin permet de visualiser en temps réel le diagramme de classes de conception.

3 Modélisation des classes

En phase de **conception**, une classe se modélise comme pour le diagramme de classes d'analyse, en ajoutant certaines informations supplémentaires liées au fonctionnement de notre logiciel :

- Ajouts d'**opérations** (méthodes et constructeurs de la classe). Pour chaque opération, on précise ses paramètres et éventuellement leurs types (**nom : type**). On précise également le **type de retour** de l'opération (sauf pour les constructeurs ou si la méthode ne renvoie rien).
- Ajout de la visibilité sur les attributs et les opérations (+ pour public, **-** pour privé et # pour protégé).
- On souligne les attributs et opérations de classe (**static**).
- On convertit en opérations les **attributs dérivés** (calculés) d'une classe provenant du diagramme de classes d'analyse.
- Si certains attributs sont en lecture seule, on peut ajouter la contrainte **readOnly**.





4 Associations entre les classes

Comme pour le diagramme de classes d'analyse, on spécifie les **associations** entre les différentes classes du diagramme. Cependant, comme nous sommes en phase de conception, il faut préciser beaucoup plus de détails sur une association.

En plus des associations simples que vous connaissez déjà, on peut aussi préciser des associations plus précises selon le contexte, comme la **composition**, l'**association qualifiée**...

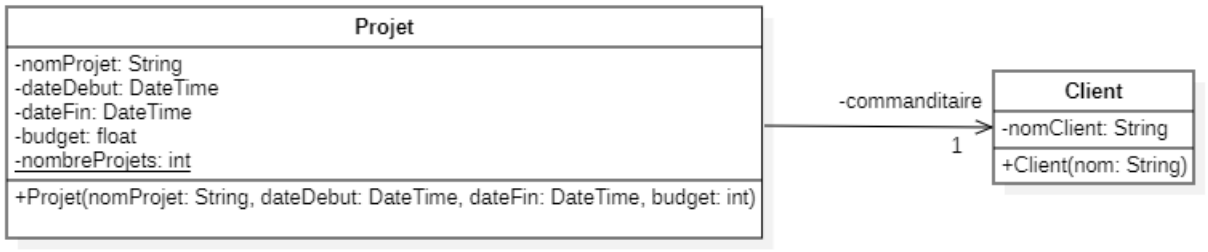
4.1 Navigabilité, attributs

Dans un diagramme de classes de **conception**, la représentation d'une association entre deux classes diffère un peu :

— On ne nomme pas l'association (on va plutôt utiliser les rôles).
— La **navigabilité** (le sens) doit être précisée (en conception les associations ne sont pas forcément bi-directionnelles, contrairement à l'analyse).
— On précise les **rôles** de chaque côté **navigable** de l'association.

Un rôle précisé sur un des côtés d'une association représente un **attribut** implicite de la classe opposée, dans l'association. On doit donc préciser la **visibilité** de ce rôle. Il ne faut donc pas lister cet attribut au niveau de la classe, il est déduit par l'association. Si l'association est bi-directionnelle, on aura donc deux rôles, et un attribut pour chaque classe.

Par exemple : dans une entreprise, chaque projet est commandé par un client. Dans le diagramme de classes ci-dessous, l'association est navigable depuis **Projet** vers **Client** en précisant le rôle **commanditaire**. La classe **Projet** possédera alors un attribut **commanditaire** (de type **Client** et **privé**, d'après le diagramme). Par contre, la classe **Client** ne possédera pas d'attributs **projets** car l'association n'est pas navigable dans le sens **Client** vers **Projet**.



Une fois traduit dans un langage de programmation orienté objet, on obtient alors le code suivant pour la classe **Projet** (ici, en **Java**) :

```java
class Projet {

    private String nomProjet;
    private LocalDate dateDebut;
    private LocalDate dateFin;
    private float budget;
    private static int nombreProjets;
    private Client commanditaire;

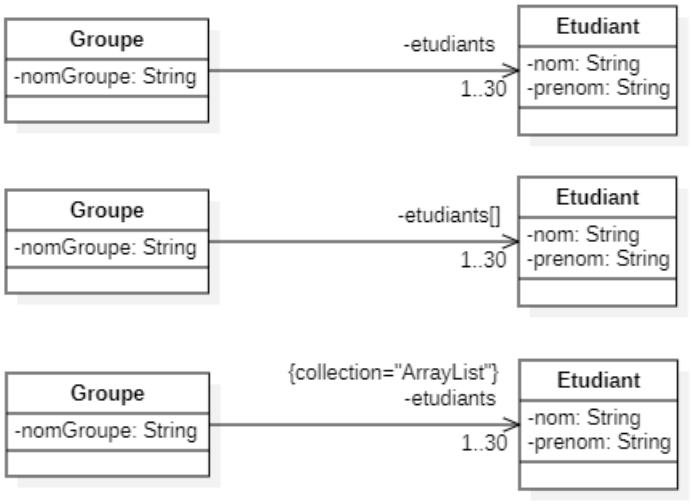
    //Méthodes
}
```

Cependant, il est aussi possible de ne pas représenter l'association et seulement utiliser en attribut en le précisant explicitement, cette fois-ci, comme attribut de la classe. Cela permet d'alléger le diagramme, mais donne beaucoup moins de précisions. On peut notamment utiliser cette notation quand on ne veut pas représenter toutes les classes, par exemple, si on souhaite afficher seulement une portion du diagramme de classes (par exemple, une couche du logiciel).

4.2 Collections

Lorsqu'on a une cardinalité multiple (plus de 1 ou bien *) sur des côtés d'une association, alors l'attribut en résultant est une **collection**. Cela signifie qu'une des classes de l'association contient un ensemble d'instances de l'autre classe.

Par défaut, si rien n'est précisé, cela sera lors de l'implémentation qu'on décidera du type de cette collection (tableau, liste, ensemble...). Cependant, si on souhaite indiquer le type de la collection au niveau de l'association, cela est possible.

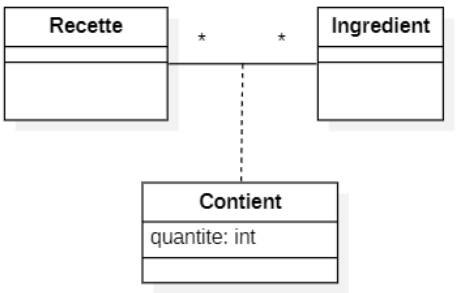


4.3 Classes-associations

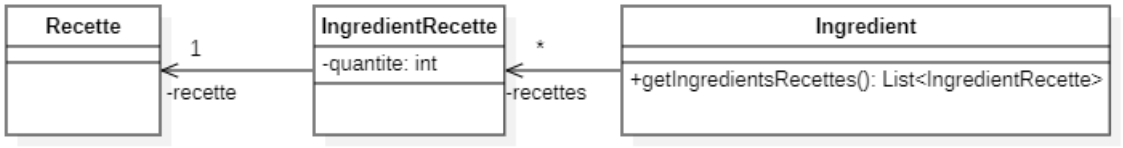
Les **classes-associations** que nous avons utilisé en phase d'analyse ne peuvent pas être implémentées dans un langage orienté objet ! Si ce concept est bien utile, notamment dans le cas d'un diagramme de classes persistantes, en conception, il faudra convertir ces classes en classes coordinatrices puis programmer les différentes contraintes d'unicité du couple d'identifiants, lors de l'implémentation.

Comme pour les autres associations, il faudra réfléchir à la navigabilité des deux associations reliées à la classe coordinatrice et également choisir où placer l'opération qui permettra de récupérer les instances de la classe coordinatrice.

Par exemple, considérons le diagramme de classes d'analyse suivant :

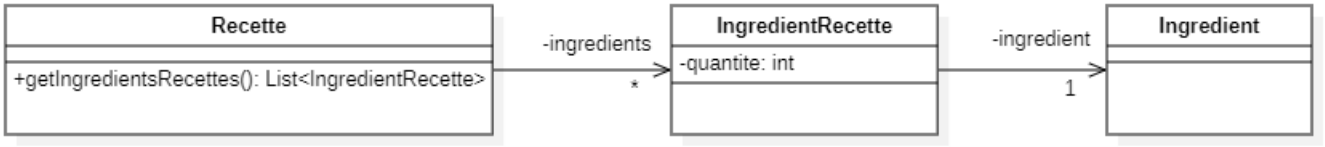


On pourrait le traduire ainsi :



Mais si on essaye de lire la logique de ce choix de conception, ce sont donc les ingrédients qui connaissent les recettes dans lesquels ils sont utilisés (et en quelles quantités)...cela ne semble pas très cohérent !

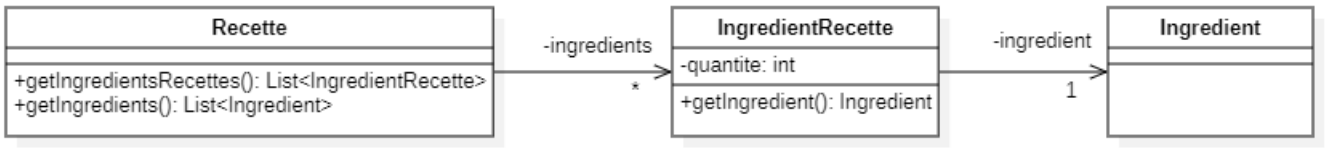
Il semble plus logique de faire l'inverse : chaque recette connaît les ingrédients dont elle a besoin (et en quelles quantités) :



Dans certains cas, il peut être intéressant d'avoir des navigabilités bi-directionnelles pour ce genre de modélisation, mais cela doit être vraiment indispensable, car cela rajoute des dépendances et des contraintes de programmation qui pourraient être évitées.

On reprendra la même logique pour les associations n-aires en ajoutant des classes liées à la coordinatrice.

Enfin, il est conseillé d'ajouter une opération (dans la classe "contenant" les autres) permettant de retrouver toutes les instances de l'autre classe.

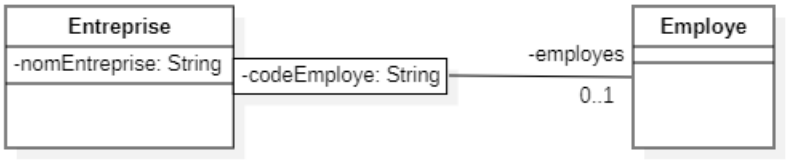


4.4 Associations qualifiées

Dans de nombreux langages de programmation orientés objet, on retrouve des structures de **collections associatives**. Par exemple, les **dictionnaires** en python, les **tableaux associatifs** en PHP, ou bien les **Map** en Java.

Ce type de structure permet d'associer une valeur "clé" à une autre valeur. Pour représenter une Map associant certaines clés à une des classes de notre programme, on utilise des **associations qualifiées**.

Dans l'exemple suivant, une entreprise possède des employés, et chaque employé possède un code unique. Donc, on peut dire que pour un code employé donné, on associe au maximum un seul employé pour cette entreprise (ou aucun si le code n'est pas attribué).




```
class Projet {
    private String nomEntreprise;

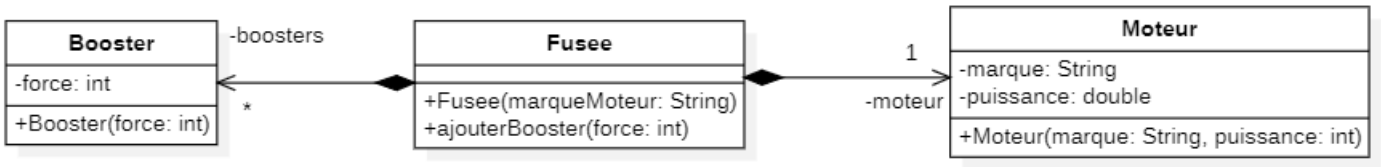
    private Map<String, Employe> employes;
}
```

4.5 Composition

Enfin, le concept de **composition** permet d'indiquer qu'une instance de la classe agrégée n'appartient qu'à une instance de la classe agrégat (à un instant t) et **ne peut pas être partagée**. Ainsi, quand la classe "propriétaire" est détruite, ses composants le sont aussi.

Si l'on souhaite que les données des composants agrégés soient lisibles à l'extérieur du conteneur (via des getters par exemple), il faut faire en sorte que les composants et leurs données ne puissent pas être altérés. Pour cela, on peut par exemple renvoyer une **copie profonde** de ce composant et pas directement l'instance du composant (ce n'est pas la même instance qui est partagée, seulement une copie avec les mêmes données).

Dans l'exemple suivant, une fusée contient un moteur et différents boosters. On a bien à faire à une **composition** : si on supprime une instance de la fusée, il n'y a aucun intérêt à garder les instances de son moteur et de ses boosters. Les instances des boosters et du moteur ne doivent pas être partagées et modifiées (directement) par d'autres instances de différentes classes (mais on peut renvoyer une copie profonde dans le cas où les informations des boosters ou du moteur aient besoin d'être lues). La fusée peut modifier les informations de ses composants et interagir avec.



On pourrait alors avoir l'implémentation suivante :

```
class Fusee {
    private Moteur moteur;
    private List<Booster> boosters;

    public Fusee(String marqueMoteur) {
        moteur = new Moteur(marqueMoteur, 5.0);
        boosters = new ArrayList<>();
    }

    public void ajouterBooster(int force) {boosters.add(new Booster(force));}
}
```

Ce qui diffère de l'implémentation suivante, qui n'est pas une composition :

```
class Fusee {
    private Moteur moteur;
    private List<Booster> boosters;

    public Fusee(Moteur moteur) {
        this.moteur = moteur;
        boosters = new ArrayList<>();
    }

    public void ajouterBooster(Booster booster) {boosters.add(booster);}
}
```

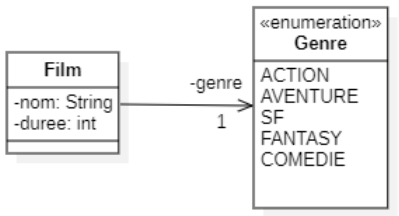
Ainsi, si un objet instancie directement certains de ses attributs (pas de passage de l'instance via les paramètres via le constructeur ou autre) et ne les partage pas, alors on peut préciser l'association sous la forme d'une composition.

De nouvelles compositions peuvent apparaître sur le diagramme dans la phase de conception, en complément de celles déjà trouvées lors de la phase d'Analyse, car de nouvelles entités et associations émergent, selon les choix effectués.

Concernant l'**agrégation blanche**, elle peut éventuellement avoir une sémantique au niveau de la conception, pour indiquer, par exemple, qu'une classe est injectée (injection de dépendance) dans une autre, un peu comme dans la deuxième implémentation proposée au-dessus. Nous en reparlerons plus amplement de ce concept en temps voulu.

5 Énumérations

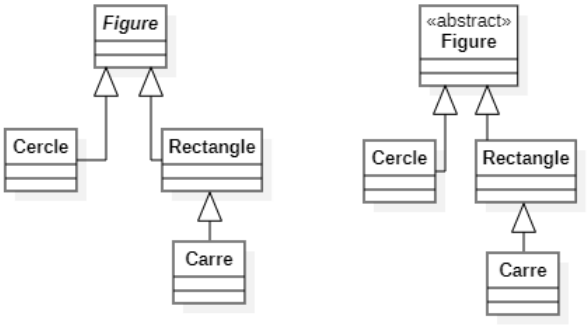
On représente une énumération en listant ses valeurs et en plaçant le **stéréotype** "enumeration" au niveau du nom de la classe.



Pour rappel, une énumération peut posséder des attributs, des constructeurs et des méthodes. Au niveau du diagramme, on peut éventuellement ajouter les opérations à l'énumération, mais on ne précisera pas les attributs.

6 Héritage

Concernant l'héritage, il se représente comme dans le diagramme de classes d'analyse. Certaines classes peuvent être **abstraites**. Dans ce cas, on indique le nom de la classe en **italique** ou bien on ajoute le stéréotype **abstract** à la classe (plus clair sur un dessin).



De même, les **méthodes abstraites** s'écrivent en italique et/ou avec le stéréotype **abstract**.

En théorie, en conception, rien n'interdit une classe d'hériter de **plusieurs classes** (héritage multiple). Cependant, il n'est pas possible de traduire ce concept dans la plupart des langages de programmation (notamment Java). Même quand cela est possible, en conception et en programmation, l'héritage multiple est fortement déconseillé et peut conduire à une architecture bancale et de mauvaise qualité. Cela peut être facilement remplacé par un système exploitant des interfaces, des designs patterns...

Bref, même si on ne pense pas encore forcément au langage qu'on va utiliser au niveau de la conception, il faut prendre en compte cet aspect ! Mais peu importe le langage, il vaut mieux s'abstenir de faire de l'héritage multiple. On s'interdira donc de l'utiliser dans nos diagrammes de classes de conception.

Attention : un héritage déterminé en phase d'analyse ne donnera pas nécessairement un héritage en phase de conception. D'ailleurs, nous verrons que dans certains cas, **la composition** (faible ou forte) **est préférable à un héritage**.

7 Interfaces

Une **interface** est une classe qui ne contient que des **signatures** de méthodes (et pas leur code). C'est un peu comme une classe abstraite qui ne contiendrait aucun attributs et seulement des méthodes abstraites.

Une classe peut implémenter **une ou plusieurs interfaces**. Dans ce cas, cette classe doit implémenter toutes les méthodes de l'interface (sauf si la classe est elle-même abstraite).

Une interface est donc un **contrat** et les classes qui l'implémentent s'engagent à remplir ce contrat en implémentant les méthodes demandées.

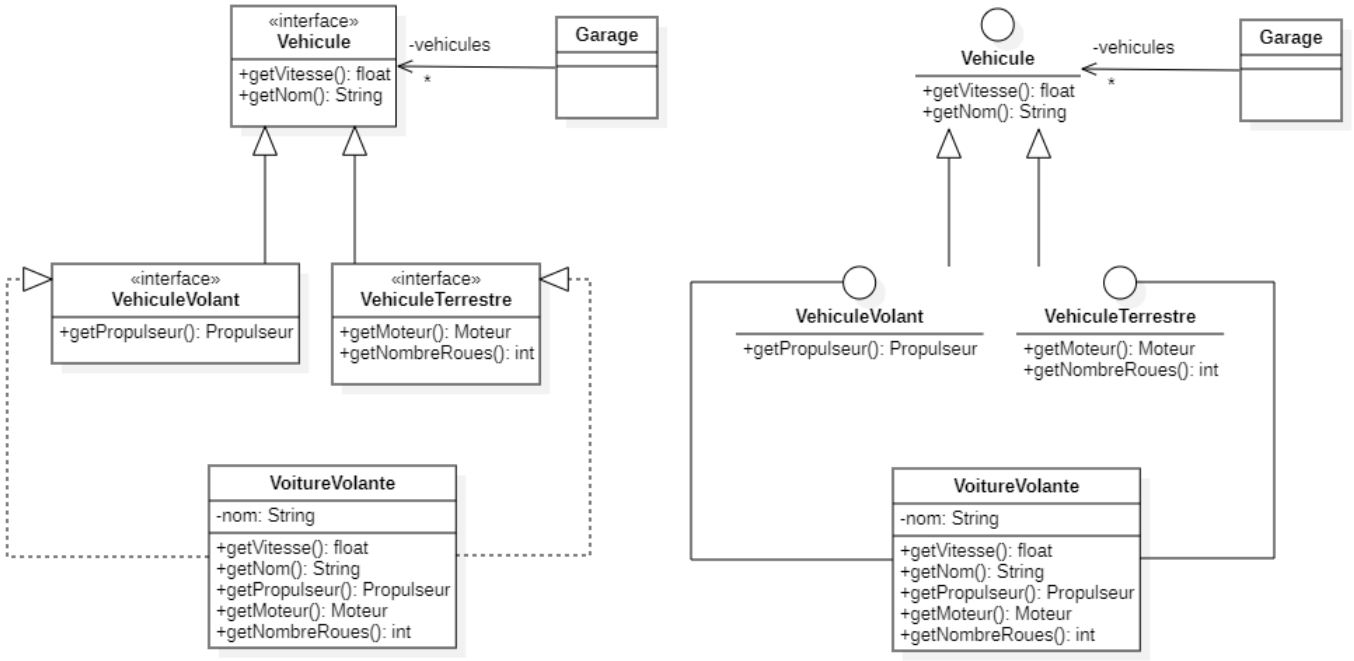
Au delà de l'aspect "contrat", une interface est aussi un **type**. Une classe implémentant une interface peut donc être vue comme une instance du type de l'interface. Cela veut dire qu'une **interface** peut tout à fait être reliée à une autre classe par une association (comme un type

normal). De plus, contrairement à l'héritage, une classe peut implémenter autant d'interfaces qu'elle souhaite !

Une interface peut également hériter (des signatures) d'une ou plusieurs autres interfaces.

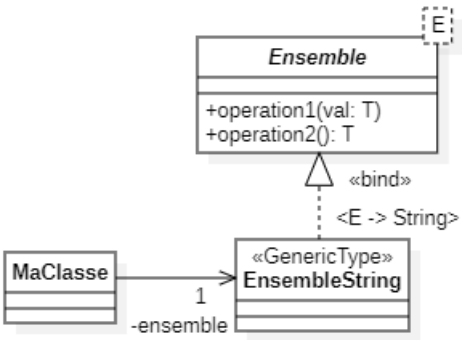
En UML, on représente une interface avec le stéréotype **interface** et on représente l'implémentation de cette interface avec une flèche similaire à celle de l'héritage, mais en pointillée.

Il est aussi possible d'utiliser la notation **lollipop** en représentant l'interface par un petit rond et l'implémentation avec un trait plein.



8 Généricité

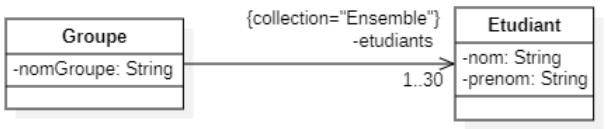
En programmation objet, certaines classes peuvent être **paramétrées** avec un ou plusieurs types (elles possèdent un ou plusieurs types générique). Si on utilise une classe générique paramétrée avec une classe spécifique (par exemple, **MaClasse<Integer>**), on représente une classe "virtuelle" spéciale liée à la classe paramétrable avec une relation similaire à celle de l'implémentation d'une interface en indiquant le stéréotype **bind** et en précisant quelle est la classe qu'on utilise comme paramètre. Une classe peut posséder plusieurs types paramétrables.




```
class MaClasse {
    private Ensemble<String> ensemble;
}
```

Note à part, si le développeur définit une nouvelle **collection** générique comme nous le faisons ici avec **Ensemble**, si on souhaite l'utiliser dans le cadre d'une association avec une cardinalité multiple, il peut être préférable de ne pas faire de lien vers notre nouvelle classe "collection"

(surtout si on l'utilise beaucoup) et plutôt l'indiquer comme type au niveau de l'association. En effet, comme nous l'avons vu, il n'est pas obligatoire de représenter tous les attributs issus de relations entre les classes par des associations.




```java
class Groupe {
    private Ensemble<Etudiant> ensemble;
}
```

9 Threads

Certaines classes généralement appelées **Thread** (processus) sont des classes qui s'exécutent de manière indépendante lors de la durée de vie du programme. Un programme possède au moins un thread principal (celui qui s'exécute au lancement du programme) mais peut également définir et exécuter différentes tâches en parallèle via d'autres classes exécutant leur propre thread. En Java, la classe **Thread** (et/ou l'interface **Runnable**) permet de faire cela.

Au niveau d'UML, les classes possédant et exécutant un thread sont appelées **classes actives** :



10 Les dépendances

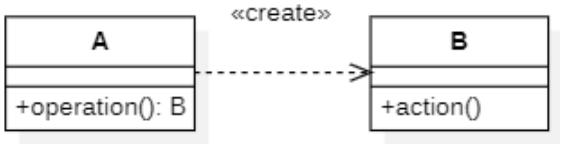
Dans un diagramme de classes de conception, un élément A est dit **dépendant** d'un élément **B** si un changement qui intervient dans **B** peut potentiellement entraîner une modification de **A**. Plus grossièrement, on peut aussi dire que **A** dépend de **B** si la suppression de **B** entraîne le dysfonctionnement de **A**. Une dépendance est représentée par une flèche en pointillée.



Il est judicieux d'ajouter des **stéréotypes** sur les dépendances pour préciser l'action en lien avec la dépendance. Par exemple **use**, de manière générique, quand une classe utilise une autre classe, qu'on peut affiner en **create** si la dépendance est liée à l'appel d'un constructeur, ou bien **call** quand on appelle une méthode d'une classe.

Exemple avec le stéréotype create

```java
class A {
    public B operation() {
        B b = new B(...);
        return b;
    }
}
```



Exemple avec le stéréotype call

```
class A {
    public void operation(B b) {
        b.action();
    }
}
```



Il existe de très nombreuses dépendances entre les classes. Certaines sont triviales et n'ont donc pas besoin d'être précisées. D'autres sont moins évidentes et peuvent être précisées, au besoin. Néanmoins, toutes les dépendances non-triviales n'ont pas forcément besoin d'être représentées. On peut se limiter à celles qu'on estime importantes (afin de garder un diagramme relativement léger).

Par exemple, une association entre deux classes implique une dépendance triviale. Il n'y a donc pas besoin de la préciser. Il en va de même pour la plupart des autres relations : héritage, implémentation d'interface, binding de classe paramétrable...

Pour ne pas surcharger le diagramme, on peut se fixer les règles suivantes :

— Si une **association** existe déjà entre deux classes, cela implique déjà une **dépendance** (de la classe possédant l'attribut vers la classe correspondant au type de l'attribut...). Si d'autres types de dépendances entre ces deux classes existent, **il n'est alors pas nécessairement utile de les préciser** (par exemple, si une classe possède un objet en attribut, elle va probablement appeler des méthodes dessus...), **sauf si elles revêtent une importance particulière dans la conception**.

— Par contre, s'il existe des dépendances entre deux classes sans pour autant qu'il y ait des associations entre elles, **il est impératif de bien préciser ces dépendances**, car elles ne sont **pas triviales**.

— Si une méthode d'une classe reçoit en paramètre un objet d'un certain type sans pour autant appeler de méthode dessus, théoriquement, il existe bien une dépendance entre les deux classes, mais elle ne porte que sur le nom de la classe (pas de dépendance *use*...). Dans ce cas, **on ne représente pas cette dépendance, car elle n'a que très peu d'impact**. L'objet est seulement en "transit" dans la classe. Par exemple, dans le cas d'interfaces (ou de méthodes abstraites) dont la signature de certaines méthodes possèdent des objets d'un certain type, il est inutile de préciser une dépendance vers la classe correspondante, car on est certain qu'il n'y a pas d'utilisation de l'objet en question...

Idéalement, lors de la conception, il faut essayer de réduire au mieux les dépendances et éviter les dépendances inutiles. Par exemple :

```
class A {
    private B b;

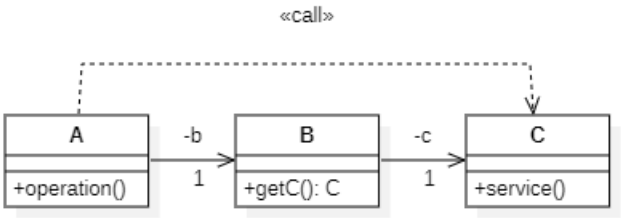
    public void operation() {this.b.getC().service();}
}

class B {
    private C c;

    public C getC() {return this.c;}
}

class C {
    public void service() {}
}
```

Dans ce code, **B** dépend de **C** et **A** dépend de **B**, mais aussi de **C** car elle appelle la méthode **service** de cette dernière.



Cette dépendance peut être évitée en suivant la loi **Démeter** qui précise qu'un objet **A** étant en relation avec un objet **B** doit pouvoir utiliser les données et les services offerts par **B** mais ne doit pas pouvoir se servir de **B** pour accéder à un troisième objet **C** et utiliser ces services. Globalement, la classe **B** ne doit pas exposer sa structure interne.

Il est toujours possible d'accéder au service voulu en ajoutant une nouvelle méthode dans **B** tout en supprimant la dépendance de **A** vers **C** :

```
class A {
    private B b;

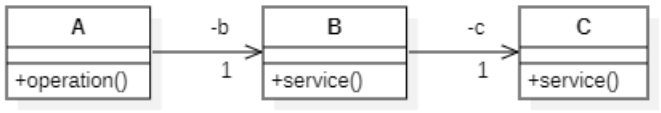
    public void operation() {this.b.service();}
}

class B {
    private C c;

    public void service() {this.c.service();}
}

class C {
    public void service() {}
}
```

Ainsi, si un changement intervient dans **C**, seule la classe **B** sera potentiellement affectée.



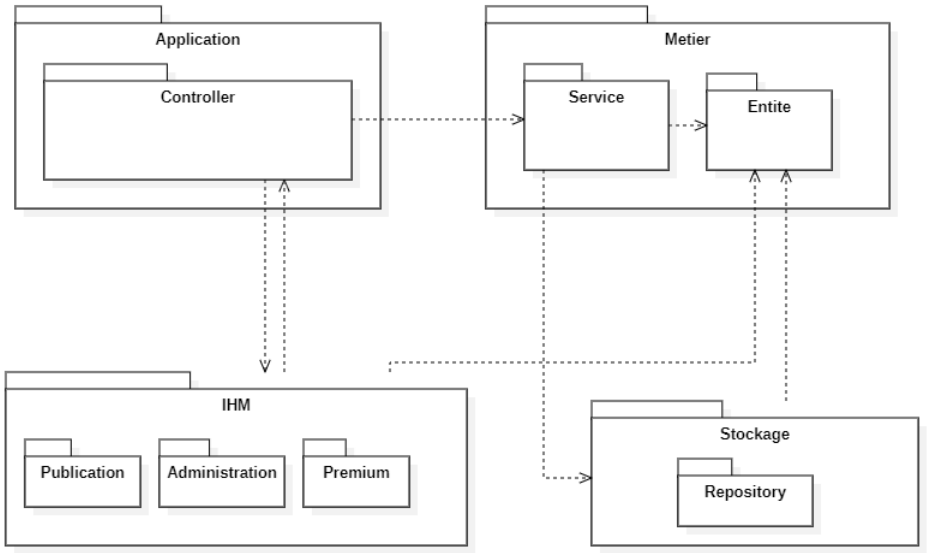
Attention, cette modélisation n'est pas souhaitable dans tous les cas. Par exemple avec la classe coordinatrice que nous avons vu plus tôt pour traduire la classe association. Mais c'est un cas bien particulier où il est clairement établi que la classe coordinatrice est une classe de transitivité entre les deux classes associées.

Il est aussi judicieux de limiter le plus possibles les dépendances bi-directionnelles qui peuvent être difficiles à maintenir, ainsi que les dépendances cycliques.

Le principe dit de **faible couplage et forte cohésion** consiste à limiter au mieux les dépendances d'une classe tout en ayant une forte cohésion à l'intérieur de celle-ci, c'est à dire qu'elle sert à réaliser une tâche claire et précise (responsabilité unique).

Enfin, il est possible de produire un **diagramme de paquetages** représentant les paquetages de l'application et leur dépendances. Un **paquetage** est un regroupement nommé de différentes classes (et de sous-paquetages). Un logiciel est organisé en différents paquetages représentants les différentes couches de l'application et permet de regrouper les classes qui ont un rôle commun et/ou collaborent.

Un paquetage **A** dépend d'un paquetage **B** quand une des classes contenues dans le paquetage **A** dépend d'une classe contenue dans le paquetage **B**. Ce diagramme permet d'avoir une vue globale de l'architecture de l'application et des interactions entre les différentes couches. Ici aussi, on peut appliquer le principe de faible couplage et de forte cohésion, mais sur les paquetages entiers.



Par ailleurs, on peut tout à fait fragmenter la conception en réalisant plusieurs diagrammes de classes de l'application (un par paquetage) puis dessiner celui de paquetage pour avoir une vision plus globale.

11 Architecture en couches

Nous l'avions évoqué au début du cours, mais l'architecture d'un logiciel se divise en différentes **couches** qui interagissent entre elles. Généralement, un logiciel est au moins composé de **trois couches** essentielles : la couche **présentation** (ou bien frontière, ou IHM), la couche **métier** et la couche **application**.

La couche **métier** est en partie définie grâce à la phase d'analyse. Elle contient généralement toutes les entités **métiers** qui sont représentées dans le diagramme de classes d'analyse. Leur but est de stocker de l'information sur les entités manipulables de l'application. On y retrouve parfois une couche de **services** qui permettent d'interagir avec ces entités et assurer la **logique de l'application** (vérifications, opérations...). C'est le coeur de l'application.

La couche **présentation** présente les données (graphiquement ou non, par exemple, juste en console) et permet une interaction avec l'utilisateur (interface-homme-machine). Cette couche peut lire les données issues de la couche métier. Généralement, via un système de programmation événementielle, l'utilisateur produit des évènements captés par l'interface ce qui provoque l'envoi de requêtes vers des contrôleurs issus de la couche **application**.

La couche **application** contenant (notamment) des classes **contrôleurs** qui permettent de faire le lien entre la couche **présentation** et la couche **métier**. L'IHM envoie des requêtes vers le contrôleur qui s'occupe de les traiter, de vérifier les données, d'ouvrir de nouvelle interfaces, d'appeler les bons service, de récupérer des entités, de les modifier, etc.

De plus, si les données sont persistantes, la couche **stockage** vient s'ajouter, contenant les **DAO** (Data Access Object) permettant de faire persister (et de charger) les données auprès du support de stockage (par exemple, en effectuant des requêtes SQL). On retrouve également des classes appelées **Repository** qui utilisent ou sont **directement fusionnées** avec les DAO et qui utilisent un langage proche du **domaine** de l'application pour ses différentes opérations.

Bref, ces différentes couches permettent de définir et de séparer les zones d'activités du logiciel. Tout l'intérêt est de faire communiquer ces couches entre elles.

La connaissance de ces couches ne donne pas encore la structure de l'application : il faut choisir une **architecture** qui permet de les structurer, de les exploiter et de définir comment elles communiquent concrètement. Il en existe plusieurs. On peut par exemple choisir l'architecture **MVC** que vous connaissez peut-être déjà grâce aux cours de développement web.

Cette architecture permet de séparer les entités, les vues et les contrôleurs de l'application et de les faire communiquer :

- La partie **modèle** (M) stocke les différentes **entités** que l'on retrouve dans la couche **métier**. On peut éventuellement y retrouver les classes d'accès aux données, comme les **DAO** et/ou les **repositories**.
- Les différentes **vues** (V) correspondent à la couche **présentation**.
- Les différents **contrôleurs** (C) gèrent la couche **application** (ils reçoivent les requêtes, déclenchent les opérations, récupèrent les données, invoquent les vues, etc).

Néanmoins, il n'est pas explicitement fait mention des **services** (ou d'autres couches) dans cette architecture. Il est généralement recommandé de **ne pas rester fixé sur les trois couches**

de base et **d'étendre le MVC** en **répartissant les responsabilités** (une couche service, une couche spécifique pour le stockage des données, etc...).

Dans une architecture MVC basique (qu'on ne fait pas évoluer), les contrôleurs vont prendre en charge le rôle des services et effectuer une grande partie de la **logique métier**. Dans certains cas, cela peut vite créer des **contrôleurs énormes** ayant beaucoup **trop de responsabilités**.

Une idée serait alors de venir placer une couche **service** entre les **contrôleurs**, les **entités** et une couche **stockage**. Ainsi, la logique métier est isolée et mieux répartie, et on obtient donc une séparation plus forte.

Le but d'un **service** est de **séparer et d'isoler certaines responsabilités** liées à la logique métier de l'application (calculs, contrôle des données, accès à la couche de stockage des données...). Dans ce type d'architecture, le **contrôleur** reçoit les requêtes, fait des vérifications simples vis-à-vis des données envoyées (pas de données manquantes, bon format, etc.), les privilèges de l'utilisateur, identifie le ou les services à utiliser, y fait appel, génère et affiche de nouvelles interfaces...Le **service** quant à lui contient la **logique métier** qui effectue des contrôles et des opérations liés aux règles de l'application et des entités, appelle la couche de stockage, etc. On sépare ainsi le **traitement de la requête** de la **logique métier** de l'application.

Par exemple, si on considère le code suivant :

```
public class ControllerProduit extends Controller {

    /*...*/

    public void modifierPrixProduit(int idProduit, double prix) {
        User user = applicationContext.getLoggedUser();
        if(user != null && user.isAdmin()) {
            Produit prod = repositoryProduit.find(idProduit);
            if(prod == null) {
                PopUpFail popup = new PopupFail("Produit introuvable!");
                popup.show();
            }
            else if(prix <= 0) {
                PopUpFail popup = new PopupFail("Le prix du produit ne peut pas être <=
                ↪  0!");
                popup.show();
            }
            else {
                prod.setPrix(prix);
                repositoryProduit.update(prod);
                PopUpSuccess popup = new PopUpSuccess("Prix mis à jour!");
                popup.show();
            }
        }
        else {
            PopUpFail popup = new PopupFail("Droits insuffisants!");
            popup.show();
        }
    }
}
```

Le contrôleur s'occupe du traitement de la requête en vérifiant les droits et en ouvrant les interfaces correspondantes, mais il s'occupe aussi de la logique métier. A la place, on pourrait alors introduire un **service** :

```
public class ControllerProduit extends Controller {

    /*...*/

    public void modifierPrixProduit(int idProduit, double prix) {
        User user = applicationContext.getLoggedUser();
        if(user != null && user.isAdmin()) {
            try {
                serviceProduit.updatePrixProduit(idProduit, prix);
                PopUpSuccess popup = new PopUpSuccess("Prix mis à jour!");
                popup.show();
            }
            catch (ServiceException e) {
                PopUpFail popup = new PopupFail(e.getMessage());
                popup.show();
            }
        }
        else {
            PopUpFail popup = new PopupFail("Droits insuffisants!");
            popup.show();
        }

    }

}

public class ServiceProduit {

    /...

    public void updatePrixProduit(int idProduit, double prix) {
        Produit prod = repositoryProduit.find(idProduit);
        if(prod == null) {
            throw new ServiceException("Produit introuvable!");
        }
        else if(prix <= 0) {
            throw new ServiceException("Le prix du produit ne peut pas être <= 0!");
        }
        else {
            prod.setPrix(prix);
            repositoryProduit.update(prod);
        }
    }

}
```

Enfin, il est aussi possible d'avoir une couche **réseau** si la nature de l'application l'impose. Il est intéressant de noter que dans le cadre d'un logiciel en réseau, ces couches ne se trouvent pas forcément sur la même application ! Les couches sont réparties ou partagées entre un programme client et un programme serveur. Généralement, la couche "service" va se trouve exclusivement sur l'application serveur, la couche "frontière" (IHM) sur le client, alors que la couche "métier"

va être dupliquée entre les deux programmes...

Il est primordial que le développeur connaisse ces couches, leur rôle, et surtout, à quelle couche appartient tel ou tel objet, et par conséquent, où ajouter tel ou tel bout de code. Ces couches garantissent une certaine indépendance entre les différents modules de l'application ce qui renforce le principe de **faible couplage** et **forte cohésion** que nous avions abordé lors du dernier cours. Par exemple, il serait impensable (et irresponsable) que la couche **présentation** (par exemple, les vues) interagisse directement avec la couche **stockage**.

Concernant les 3 couches principales, même si la forme varie et évolue, on les retrouve au coeur de différents types d'architectures : MVC (Model-View-Controller), ECB (Entity-Control-Boundary), MVC2, et les diverses architectures de **Framework**.

Il n'y a pas de règles précise quant à l'utilisation de telle ou telle architecture, mais il faut que les différentes couches essentielles soient représentées et bien organiser les rôles de chaque composant, et introduire de nouvelles couches au besoin, afin de correctement séparer les responsabilités.

Dans le cas des futurs TPs, nous allons plutôt tendre vers une architecture **MVC** qui se développera en **plusieurs couches**, notamment en introduisant une couche **service**.

12 Qualité de la conception

12.1 Les principes SOLID

SOLID est un acronyme désignant les 5 principes clés de **conception** à mettre en pratique lors du développement d'un logiciel :

- Le principe de responsabilité unique (**S**ingle responsability)
- Le principe ouvert/fermé (**O**pen/Close)
- Le principe de substitution de Liskov (**L**iskov substitution)
- Le principe de ségrégation des interfaces (**I**nterface segregation)
- Le principe d'inversion des dépendances (**D**ependency inversion)

La mise en pratique de **SOLID** couplé à une **architecture cohérente** permet d'obtenir un **code qualitatif** rendant l'application développée facilement **compréhensible**, **évolutive** et **extensible**.

12.2 Les designs patterns

Un **design pattern** est une solution **réutilisable** et **adaptable** vis-à-vis d'un problème de **conception** logicielle assez récurent.

Quand on parle de **design patterns** on fait généralement référence aux patterns dits **GoF** (Gang of Four) présentés dans le livre **Design Patterns : Elements of Reusable Object-Oriented Software**, qui est une référence dans le monde de l'ingénierie logicielle

Un pattern est généralement présenté via un **diagramme de classes de conception** et un exemple d'implémentation. Un pattern a pour but d'être applicable et adaptable à n'importe quel projet ou contexte.

Tous ces patterns permettent de **renforcer l'application des principes SOLID**. Par ailleurs, certains langages comme Java proposent (dans leur librairie standard) des classes et interfaces permettant d'implémenter certains patterns.

12.3 Les frameworks

Les **frameworks** sont des **infrastructures logicielles** qui proposent un **cadre de développement** au développeur.

Un framework est généralement constitué de différents **composants** organisés de manière **cohérente** afin de proposer une **architecture logicielle** (en couches), qui garantit donc une certaine qualité à la **structure** des applications produites avec cet outil.

Les frameworks englobent et implémentent généralement différents **design patterns** et "forcent" le développeur (de par leur structure et l'architecture) à respecter un certain niveau de qualité dans la conception (parfois, sans qu'il s'en rende compte).

Les étudiants du parcours RACDV vont développer un framework PHP lors du semestre 4, et les étudiants du parcours RACDV et IAMSI utiliseront des frameworks professionnels lors du semestre 5 (**Symfony**, **Vue.js**).

13 Ressources supplémentaires

Pour approfondir vos connaissances dans les notions abordées en cours de qualité de développement, quelques livres de référence :

- **UML Distilled** (2003) de **Martin Fowler**.
- **Clean code** (2008) de **Robert C. Martin**.
- **Refactoring** (2018) de **Martin Fowler**.
- **Design Patterns : Elements of Reusable Object-Oriented Software** (1994) de **Erich Gamma**, **Richard Helm**, **Ralph Johnson** et **John Vlissides**.